

NumPy - Day 1: Detailed Notes

Data Science with NumPy & Pandas (5-Day Series) | Foundations of Numerical Computing

1. Introduction to NumPy

NumPy (Numerical Python) is the core library for numerical computing in Python. Its central object is the **ndarray** (n-dimensional array) - a grid of values where **every element has the same data type**. This "one dtype" rule is exactly what makes NumPy fast: the data sits in one continuous block of memory and is processed by compiled C code.

Every major data library - **Pandas, Scikit-learn, TensorFlow, PyTorch** - is built on top of NumPy, so mastering arrays is the foundation of everything that follows in this course.

	Python list	NumPy array
Speed	Slow (interpreted loop per element)	Fast (compiled C, whole-array operations)
Memory	High (each element is a full Python object)	Low (raw values packed together)
Data types	Mixed types allowed	One dtype for the whole array
Math behaviour	<code>[1, 2] + [3, 4]</code> gives <code>[1, 2, 3, 4]</code> (join)	<code>a + b</code> adds element-wise

```
import numpy as np      # "np" is the universal alias - always import it this way
```

2. Creating Arrays

2.1 From Python lists: `np.array()`

```
a = np.array([10, 20, 30])      # 1D array (vector)
m = np.array([[1, 2, 3],
               [4, 5, 6]])      # 2D array (matrix): 2 rows x 3 columns
```

In machine learning, data always lives in 2D form: **rows = samples** (students, customers, images) and **columns = features** (marks, age, pixels).

COMMON TRAP: NumPy **silently converts** mixed types to one dtype - it never raises an error. `np.array([1, 2, "three"])` turns everything into strings, and `np.array([1, 2.5])` turns everything into floats. Also, nested lists must be rectangular: `np.array([[1, 2], [3, 4, 5]])` fails.

2.2 Array generators: `zeros`, `ones`, `full`

```
np.zeros((2, 3))      # 2x3 matrix of 0s
np.ones(5)            # 1D array of five 1s
np.full((3, 3), 7)    # 3x3 matrix filled with 7
```

COMMON TRAP: The shape must be passed as a **tuple**. `np.zeros(3, 4)` raises a `TypeError` - the correct call is `np.zeros((3, 4))`. This is the single most common beginner error.

NOTE: The output shows `0.` with a decimal point because the default dtype of these generators is **float64**, not **int**.

2.3 Sequences: `arange` vs `linspace`

```
np.arange(1, 11, 2)      # [1 3 5 7 9]          start, STOP EXCLUDED, step
np.linspace(0, 10, 5)   # [0. 2.5 5. 7.5 10.]  start, STOP INCLUDED, count
```

	np.arange	np.linspace
3rd argument	step size	number of values (count)
Stop value	EXCLUDED	INCLUDED
Best for	integer sequences	float sequences

COMMON TRAP: Avoid float steps in arange: `np.arange(0, 1, 0.1)` can produce an unexpected extra or missing value due to floating-point precision. Rule: **integers - arange; floats - linspace.**

2.4 Identity matrix: np.eye(n)

```
np.eye(4)                # 4x4 matrix, 1s on the diagonal, 0s elsewhere
```

2.5 dtype and astype()

```
np.array([1, 2, 3], dtype=float)  # force float at creation
arr.astype(int)                   # convert an existing array
```

COMMON TRAP: `astype(int)` **truncates**, it does not round: `np.array([3.9, 7.8]).astype(int)` gives `[3 7]` - the decimal part is simply cut off.

3. Array Attributes

```
m = np.array([[1, 2, 3],
              [4, 5, 6]])
m.shape      # (2, 3)  -> rows, columns
m.ndim       # 2       -> number of dimensions
m.size       # 6       -> total elements
m.dtype      # int64    -> data type
```

These are **attributes**, not methods - no parentheses. `m.shape()` raises an error. In practice you will check `.shape` constantly: **most ML bugs are shape bugs.**

COMMON TRAP: A 1D array of 5 elements has shape `(5,)` - **not** `(5, 1)`. `(5,)` means "just 5 values, no row/column structure", while `(5, 1)` is a real 2D column with 5 rows. ML libraries usually demand the 2D form - on Day 2, `reshape(-1, 1)` solves exactly this.

NOTE: `len(arr)` returns only the **first dimension**: for a `(3, 4)` matrix, `len` is 3 while `.size` is 12.

4. Indexing and Slicing

4.1 Indexing a 1D array

```
arr = np.array([10, 20, 30, 40, 50])
# index:         0   1   2   3   4
# negative:      -5  -4  -3  -2  -1
arr[0]           # 10 (first)
arr[-1]          # 50 (last - no need to know the length)
```

4.2 Slicing: arr[start : stop : step]

```
arr[1:4]      # [20 30 40]    start INCLUDED, stop EXCLUDED
arr[:3]       # [10 20 30]   from the beginning
arr[2:]       # [30 40 50]   to the end
arr[::2]      # [10 30 50]   every 2nd element
arr[::-1]     # [50 40 30 20 10] reversed
```

NOTE: The stop being excluded is consistent with `np.arange` and with Python itself - one rule everywhere.

4.3 Indexing and slicing a 2D array

```
m[1, 2]       # single value: row 1, column 2
m[1]          # entire row 1
m[:, 2]       # entire column 2
m[0:2, 1:3]   # sub-matrix: rows 0-1, columns 1-2
```

Read the comma as: **[which rows, which columns]**.

COMMON TRAP: `m[0][1]` also works, but always prefer `m[0, 1]` - selecting a full column (`m[:, 2]`) is only possible with the comma form, and it is faster.

4.4 Boolean indexing (filtering)

```
marks = np.array([45, 82, 67, 33, 91])
marks > 60      # [False True  True False  True]
marks[marks > 60] # [82 67 91] - only the True positions
```

This is the foundation of data filtering in Pandas (Day 3-4): a condition produces a boolean array, and that boolean array selects the rows.

COMMON TRAP: For two conditions, use `&` / `|` with **parentheses**: `marks[(marks > 50) & (marks < 80)]`. Writing `and/or` or a chained `50 < marks < 80` raises: "The truth value of an array is ambiguous".

COMMON TRAP: A slice is a **view**, not a copy - modifying the slice modifies the original array. Use `arr[1:4].copy()` when you need an independent copy. (Full discussion on Day 2.)

5. Element-wise Operations

```
marks = np.array([40, 55, 62])
marks + 5      # [45 60 67]    scalar applied to every element
a + b         # element-by-element (shapes must match)
marks > 50     # [False True True] comparison gives a boolean array
```

Applying one operation to the whole array at once - with no Python loop - is called **vectorization**. On Day 2 we will measure exactly how much faster it is than a loop.

COMMON TRAP: For 2D arrays, `A * B` is **element-wise** multiplication - it is NOT the mathematical matrix product. Real matrix multiplication uses `@` or `np.matmul` (Day 2). Confusing these two is the classic linear-algebra mistake.

6. The np.random Module

Function	What it does
<code>np.random.seed(n)</code>	Fixes the randomness so every run gives the same numbers (reproducibility).
<code>np.random.rand(d0, d1)</code>	Random floats, uniform between 0 and 1.
<code>np.random.randn(d0, d1)</code>	Random floats from a normal distribution (mean 0, std 1) - negative values appear too.
<code>np.random.randint(low, high, size)</code>	Random integers; high is EXCLUDED .
<code>np.random.choice(arr, size)</code>	Randomly picks elements from an existing array.
<code>np.random.shuffle(arr)</code>	Shuffles the array in place and returns None.

Why seed? ML experiments must be repeatable - without a fixed seed, you, your teammate and your notebook tomorrow all see different "random" numbers, and debugging becomes impossible.

COMMON TRAP: A dice roll needs `np.random.randint(1, 7)` - not `(1, 6)` - because the high value is excluded.

COMMON TRAP: `arr = np.random.shuffle(arr)` destroys your data: shuffle works in place and returns None, so `arr` becomes None. Call it as `np.random.shuffle(arr)` alone.

NOTE: The seed must run **before every generation** (or in the same cell). Re-running only the generation cell in Colab produces new numbers - a very common source of confusion.

7. Basic Statistics

Function	Meaning
<code>arr.min() / arr.max()</code>	Smallest / largest value
<code>arr.sum()</code>	Total of all elements
<code>arr.mean()</code>	Average
<code>np.median(arr)</code>	Middle value (function only - see trap below)
<code>arr.std() / arr.var()</code>	Standard deviation / variance (spread of the data)
<code>arr.argmin() / arr.argmax()</code>	INDEX (position) of the min / max value

argmax vs max: `max` tells you the topper's marks; `argmax` tells you the topper's roll number (index). Value vs position.

COMMON TRAP: `median` exists only as a function: `arr.median()` raises an `AttributeError` - you must write `np.median(arr)`. Most other statistics work both ways (`arr.mean()` or `np.mean(arr)`).

7.1 The axis concept (very important for ML)

```
m = np.array([[1, 2, 3],
               [4, 5, 6]])
m.sum(axis=0)  # [5 7 9]    down the rows -> one answer PER COLUMN
m.sum(axis=1)  # [6 15]    across columns -> one answer PER ROW
```

Memory rule: **the axis you pass is the one that collapses**. Pass `axis=0` and the rows collapse, leaving a per-column result. In a dataset (rows = students, columns = subjects): `axis=0` gives

per-subject statistics, `axis=1` gives per-student statistics.

NOTE: NumPy's `std` uses the population formula (`ddof=0`) while Pandas uses the sample formula (`ddof=1`) - so on Day 3, `describe()` will show a slightly different std. Expected, not a bug.

8. Quick Revision

- `np.array` / `zeros` / `ones` / `full` / `arange` / `linspace` / `eye` - creation; shape goes in as a tuple.
- `arange` excludes the stop, `linspace` includes it; `linspace`'s 3rd argument is a count, not a step.
- `astype` truncates floats; mixed input types are silently upcast.
- `shape (5,)` vs `(5, 1)`; attributes have no parentheses; `len` gives only the first dimension.
- Slicing: start included, stop excluded; `[::-1]` reverses; 2D form is `[rows, columns]`.
- Boolean indexing filters by condition; use `& |` with parentheses for multiple conditions.
- `*` is element-wise, not matrix multiplication.
- `randint` excludes high; `shuffle` is in-place and returns `None`; seed before every generation.
- `np.median` is function-only; `argmax` gives the position; the axis you pass collapses.